Databases – Exercises
Conceptual Modelling & Relational Model

Vrije Universiteit Amsterdam

General remarks: When creating a conceptual model, try to stay as close as possible to the described scenario. If an entity set does not have a key, then

(a) Check if it can be a weak entity set, that is, whether it has a discriminator that uniquely identifies it in combination with an identifying entity set.

(b) If there is no good key and no good choice for a discriminator plus identifying entity set, then consider introducing an artificial identifier (like a *customer id*).

Annotate your relations with cardinalities whenever possible!

When translating the conceptual model into a relational model, model the cardinalities as good as possible by eliminating tables and making use of **not null** and **unique** constraints.

We want to see that you understand the impact of your design choices. So describe what assumptions you have made and what cannot be modelled by your design (also for the relational model).

1. **Preschool**

You are asked to create a management system for preschools:

All preschools have a unique name, a telephone number and an address. Every preschool has one principal. Employees are persons. Every person is assigned a unique identifier and we store first and last name, the date of birth, the gender and address. For employees we additionally record the social security number. Every employee works for precisely one preschool. Addresses are stored in a separate table. They consist of street name, house number, zip code and city. Each address is assigned a unique identifier. Also job functions, such as secretary, helper and teacher, are stored in a separate table. Every job function has a unique title and a salary scale. We want to store which function could be filled by which employee. Note that an employee could be qualified for multiple functions.

Every preschool consists of at least 2 groups. Each group consists of at least 10 and at most 25 children. Every group has an identifier that is unique within the preschool. For every group we store the room and age group (a certain minimum age to a maximum age) that this group is suitable for. Every employee works for precisely one preschool, but within this preschool he/she can work for different groups. For the salary payment it is necessary to store for every employee, how many hours per week he/she has worked in what position and in which group. Children belong to precisely one preschool group; they are also persons. For every child we store an up-to-date picture. For children with siblings we also record who are their siblings. To reach parents or grand parents in case of emergencies, we store for every child a list of telephone numbers. For each child, these emergency phone numbers are ordered by importance, and we store the name of the person corresponding to the phone number.

For every child there is the choice of different care programs: e.g. there is the 'morning program' where the child visits preschool from 7 to 12 o'clock, and there is the 'half day' program from 7 to 14 o'clock. Every care program has a unique name, and we store the begin and the end time. Moreover, we store which child is assigned to which care program. Every child can be assigned to only one care program at a time. However, is possible to change the program per month and year. For example it should be possible to express that a child visits the care program 'morning' from January to March and 'half day' from April to December.

(a) Provide a conceptual database model in the form of an **entity relationship diagram**. Indicate all key and cardinality constraints. Express cardinalities using the *min*...*max* notation. Explain important **design choices** and document relevant assumptions.

(b) Create a **relational schema** for the conceptual model from item (a):

- Give the relations $R(A_1, A_2, \ldots)$ with their attributes $A_1, A_2, \ldots$.
- Underline the primary keys and and indicate the foreign key relations.

Design your relational schema (eliminate tables if necessary) such that the **cardinality constraints** from the entity relationship diagram are enforced as well as possible.

Annotate constraints that a relational database could check (e.g. nullable and unique). Explain which constraints from the conceptual model cannot be expressed in the relational model.

2. **Exam Scheduling**

Consider a university database for the scheduling of exams in classrooms.

- For the rooms, we store their room number, capacity and the building in which they are located. For each building we store a name and address (the names of the buildings are unique).
- For the courses, store their name and the course number. Each course consists of at least one section. The sections have section numbers that are unique within the corresponding course.
- Each exam concerns one or more sections of a course. Each exam has a unique id and we store the starting time, duration and room in which the exam is held.

(a) Provide a conceptual database model in the form of an **entity relationship diagram**. Indicate all key and cardinality constraints. Express cardinalities using the *min*...*max* notation (like in UML).

Explain the most important **design choices** and document relevant assumptions.

(b) Create a **relational schema** for the conceptual model from item (a):

- Give the relations $R(A_1, A_2, \ldots)$ with their attributes $A_1, A_2, \ldots$.
- Underline the primary keys and and indicate the foreign key relations.

Design your relational schema (eliminate tables if necessary) such that the **cardinality constraints** from the entity relationship diagram are enforced as well as possible.

Annotate constraints that a relational database could check (e.g. nullable and unique). Explain which constraints from the conceptual model cannot be expressed in the relational model.

3. **Library Database**

Consider the following case. You are building a simplified library database:

- The library owns (physical) *books* that are stored on shelves and loaned to customers. Books have an *ISBN number*, *title* and *publication date*. They are published by a *publisher* (which has a *name* and *phone number*). Books are written by one or more *authors* (who have a *first name*, *last name* and *year of birth*). In case of a book written by multiple authors, their *order* matters.
- Each of the books is represented by a *catalogue entry*; think of an old-fashioned card file as a model of this. There is only one 'title card' for each book in the catalogue, but there can be many physical copies of that book on the shelves. Call the title card entity set *CatalogueEntry* and the physical book entity set *BookOnShelf*. For each physical copy of a book the library keeps a *copy number* and the *date on which that copy was acquired* by the library.
- The library *loans* books to *customers*, who have a *first name*, *last name*, and a *phone number*. For each loaned book, the library keeps a record (which is also kept after the book is returned). This loan record stores the *scanner ID* of the scanner used to checkout the book, the *date when the book was loaned* and a *due date* is set for the book to be returned. When the book is in fact returned, the *return date* is also stored.

(a) Provide a conceptual database model in the form of an **entity relationship diagram**. Indicate all key and cardinality constraints. Express cardinalities using the *min*...*max* notation (like in UML).

Explain the most important **design choices** and document relevant assumptions.

(b) Create a **relational schema** for the conceptual model from item (a):

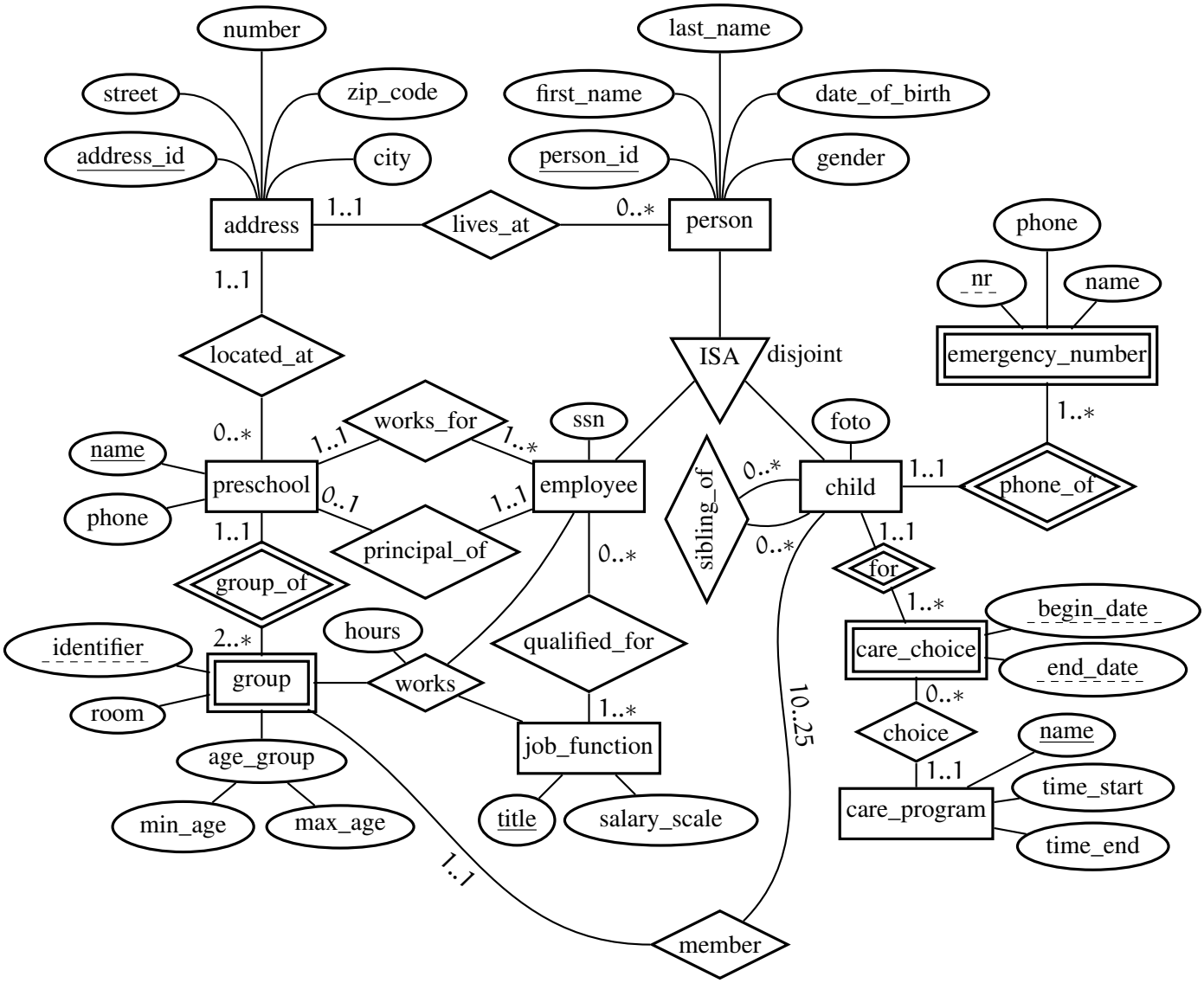
- Give the relations $R(A_1, A_2, \ldots)$ with their attributes $A_1, A_2, \ldots$.
- Underline the primary keys and and indicate the foreign key relations.

Design your relational schema (eliminate tables if necessary) such that the **cardinality constraints** from the entity relationship diagram are enforced as well as possible.

Annotate constraints that a relational database could check (e.g. nullable and unique). Explain which constraints from the conceptual model cannot be expressed in the relational model.

Solution for Preschool (1)

(a)



Important design choices:

- Every entity should have a key, every weak entity should have a discriminator.
- There should be a separate entity set for addresses.
- There should be two relations between preschools and employees:
  (a) to indicate what preschool an employee works for
  (b) to indicate which employee is the principal of each preschool

  The cardinalities should express that each employee is assigned to precisely one preschool, and that each preschool has precisely one principal.
- There should be an entity for job functions with a 'qualification' relation to employees. The cardinalities should express that every employee is qualified for at least one job function.

- The groups should be represented by a weak entity set with the preschool as identifying entity set (since the identifier for the groups is unique only within a preschool). The cardinalities of the identifying relationship should express that each group belongs to precisely one preschool and each preschool has at least 2 groups.
- There should be a relation between groups and children expressing that each child belongs to precisely one group and each group consists of 10 to 25 children.
- There should be a ternary relationship with an attribute 'hours' between employees, groups and job functions. This relation is needed to record which employee has worked how many hours per week in which group and in which job function. (Note that the attribute must be an attribute of the relation and not one of the entities.)
- The emergency numbers must be a separate entity set, preferably a weak entity set (with identifying entity set child) since this allows to express that the attribute 'no' (which indicates the order of importance) is unique for each child.
- The assignment of children to care programs are best modelled as a weak entity set; every care choice is related to precisely one child and one care program. This can either be done by two binary relations or one ternary relation. However, since the semantics of cardinalities is less clear for ternary relations it is easier to model it by two binary relations.

  An alternative to having 'care choice' as a (weak) entity would be to have a binary relation 'care choice' between children and care programs. However, then the time period (month and year) must be a *multi-valued* attribute of the relation; otherwise we can have only one time period for every child and care choice.

(b) **Basic translation:**

Here, we only eliminate tables of identifying relationships of weak entities.

- Person (person_id, first_name, last_name, date_of_birth, gender)
- Employee (person_id → Person, ssn)
- Child (person_id → Person, foto)
- Address (address_id, street, number, zip_code, city)
- Preschool (name, phone)
- Group (name → Preschool, identifier, room, min_age, max_age)
- Emergency_number (person_id → Child, nr, phone, name)
- Job_function (title, salary_scale)
- Care_choice (person_id → Child, begin_date, end_date)
- Care_program (name, time_start, time_end)

- Lives_at (address_id → Address, person_id → Person)
- Located_at (address_id → Address, school → Preschool)
- Works_for (worker → Employee, school → Preschool)
- Principal_of (principal → Employee, school → Preschool)
- Sibling_of (sibling_a → Child, sibling_b → Child)
- Qualified_for (worker → Employee, job_title → Job_function)
- Works (worker → Employee, job_title → Job_function
  (school, group_id) → Group, hours)
- Choice ((child_id, begin_date, end_date) → Care_choice, care_name → Care_program)
- Member (child_id → Child, (school, group_id) → Group)

Nullable and uniqueness constraints:

- Basically all attributes are **not null**. Except maybe for attributes that might not be known immediately, like foto for the entity set child.
- The following attributes must be declared unique:
  - *person_id* of *Employee*
  - *ssn* of *Employee*
  - *person_id* of *Child*

  since we do not want several employees or children representing the same person, and social security numbers are unique.
- We can model the cardinality constraints 'at most one' using the following **unique** constraints:
  - *person_id* of *Lives_at*
  - *school* of *Located_at*

  - *worker* of *Works_for*
  - *principal* of *Principal_of*
  - *school* of *Principal_of*
  - *child* of *Member*

Note that:

- The ternary relationship 'works' translates to table with 4 columns since the key of 'group' is the 'identifier' together with the key of 'preschool'. Likewise the binary relationship 'member' translates to a table with 3 columns.

Optimised translation:

The optimised translation has two advantages:

(a) we eliminate unnecessary tables

(b) we can model 'precisely 1' constraints

We eliminate the following tables:

- *Lives_at* by extending *Person* with the key of *Address*
  - we use the constraint *not null* to model that every employee has 1..1 address
- *Located_at* by extending *Preschool* with the key of *Address*
  - we use the constraint *not null* to model that every preschool has 1..1 address
- *Works_for* by extending *Employee* with the key of *Preschool*
  - we use the constraint *not null* to model that every employee works at 1..1 preschools
- *Principal_of* by extending *Preschool* with the key of *Employee*
  - we use the constraint *not null* and **unique** to model that every preschool has 1..1 principals, and every employee is principle of 0..1 preschools.
- *Choice* by extending *Care_choice* with the key of *Care_program*
  - we use the constraint *not null* to model that every *care_choice* corresponds to 1..1 *care_programs*
- *Member* by extending *Child* with the key of *Group*
  - we use the constraint *not null* to model that every child belongs to 1..1 groups

The resulting database schema is:

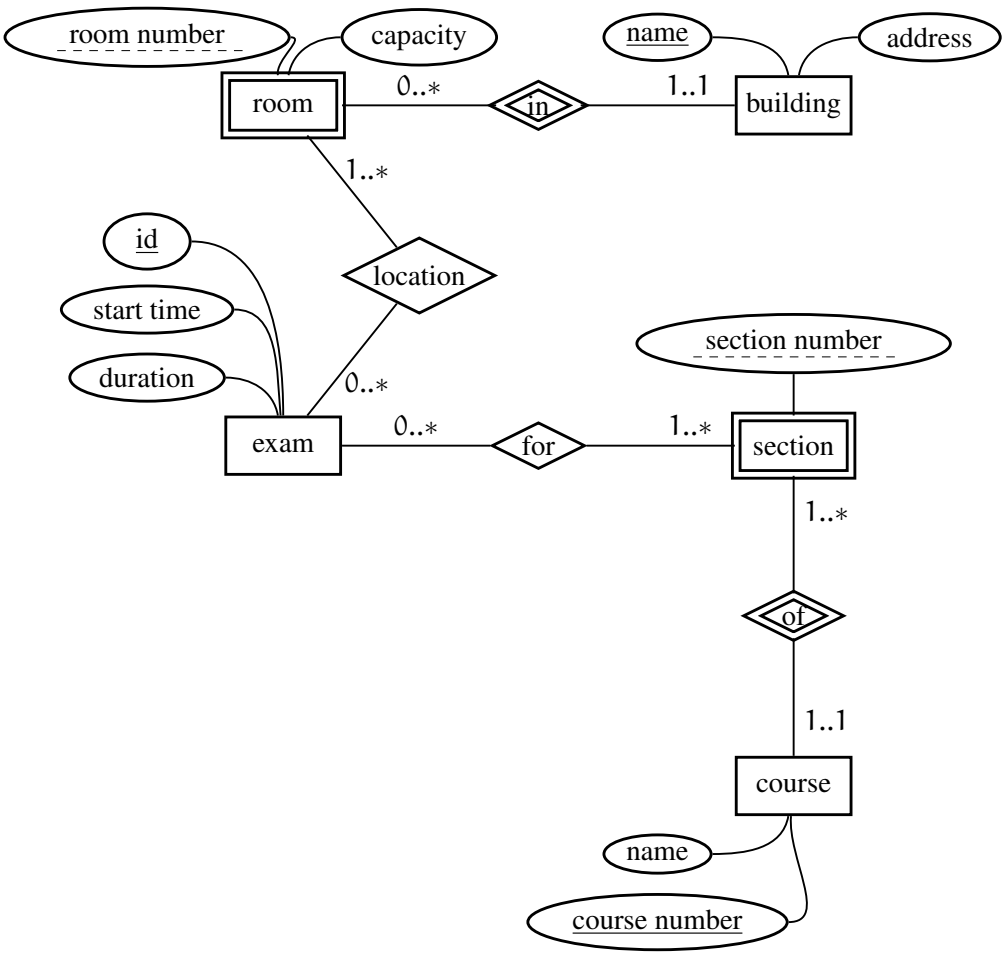
- Person (person_id, first_name, last_name, date_of_birth, gender, lives_at $\rightarrow$ Address)
- Employee (person_id $\rightarrow$ Person, ssn, school $\rightarrow$ Preschool)
- Child (person_id $\rightarrow$ Person, foto, (school, group_id) $\rightarrow$ Group)
- Address (address_id, street, number, zip_code, city)

- Preschool (name, phone, located_at → Address, principal → Employee)
- Group (name → Preschool, identifier, room, min_age, max_age)
- Emergency_number (person_id → Child, nr, phone, name)
- Job_function (title, salary_scale)
- Care_choice (person_id → Child, begin_date, end_date, care_name → Care_program)
- Care_program (name, time_start, time_end)

- Sibling_of (sibling_a → Child, sibling_b → Child)
- Qualified_for (worker → Employee, job_title → job_function)
- Works (worker → Employee, job_title → Job_function
  (school, group_id) → Group, hours)

Solution for Exam Scheduling (2)

(a)



Design choices: The room is weak entity set depending on the identifying entity set building. The room number identifies the room only in combination with the building that the room is in.

The section is a weak entity set since the section number is unique only within a certain course, thus section depends on the identifying entity set course.

We have chosen that a building can have $0..*$ or more rooms (choosing $1..*$ is also fine), allowing a building to (temporarily) have no rooms (maybe due to renovation).

Our design allows sections without any exams (choosing cardinality $1..*$ is also fine).

(b)

- Building(name, address)
- Room(roomNumber, capacity,
  inBuilding → Building) (elimination of *in* table)
- Exam(id, start time, duration)
- Course(courseNumber, name)
- Section(sectionNumber,
  of → Course) (elimination of *of* table)
- Location(id → Exam, (roomNumber, inBuilding) → Room)
- For(id → Exam, (sectionNumber, courseNumber) → Section)

To enforce the relation cardinalities by design, we have eliminated almost all relationship set tables. The **constraints** are:

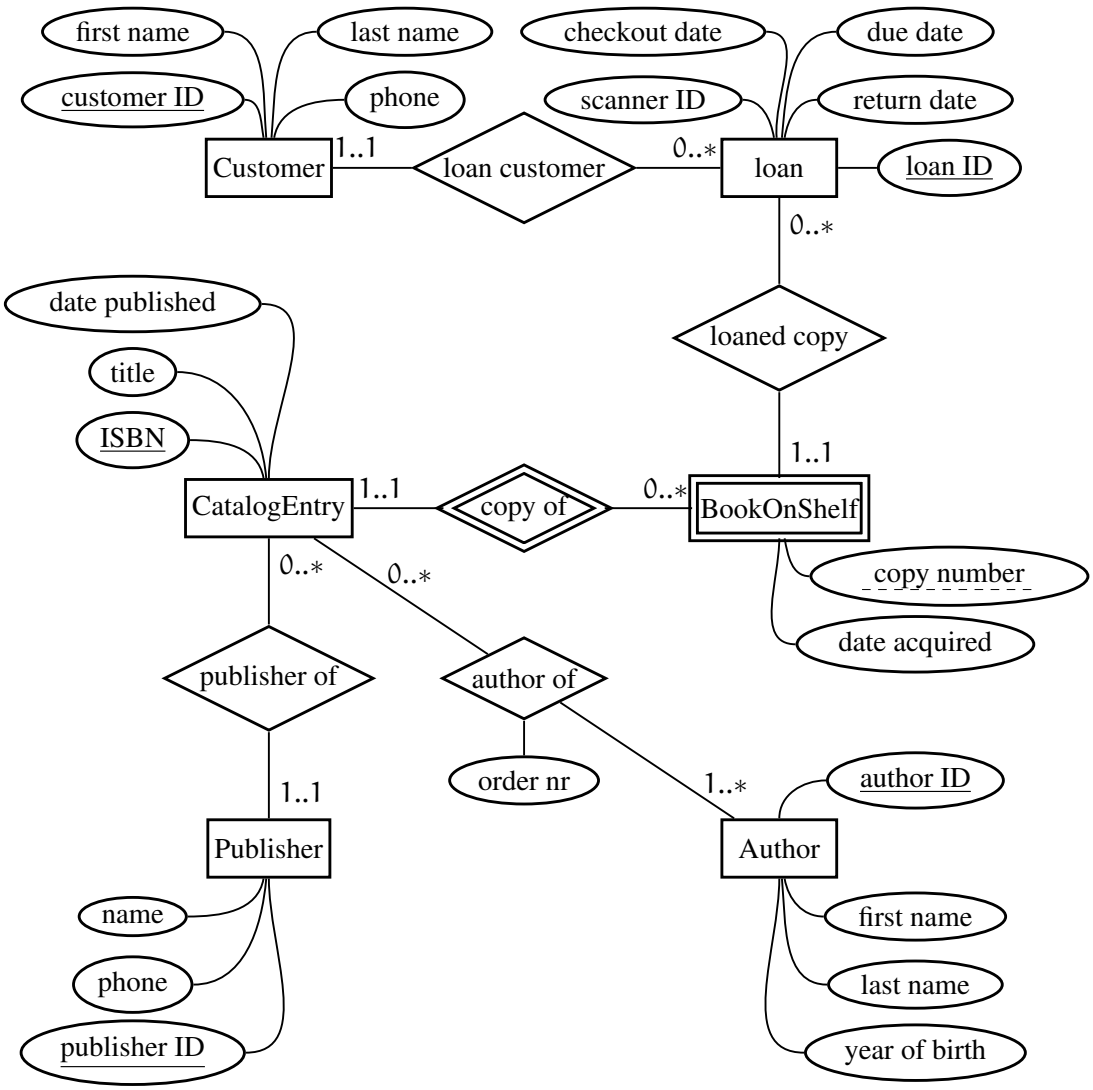
- all attributes can be declared **not null**.

The database schema above enforces all cardinality constraints except:

- a course can have 0 sections (0..∗ instead of 1..∗),
- an exam can be located in 0 rooms (0..∗ instead of 1..∗),
- an exam can have 0 sections (0..∗ instead of 1..∗).

Solution for Library Database (3)

(a)



Design choices: In this design we have tried to stay as close as possible to the scenario description while still guaranteeing that every entity set has a primary key. For author, publisher and customer, we have introduced artificial attributes (author ID, publisher ID, customer ID) as a key since combinations of name and phone are not unique and prone to be updated. Likewise, for the loan entity set we have introduced an artificial loan ID since there is no obvious choice for a discriminator and identifying entity set.

The BookOnShelf is a weak entity set as it does not have a key itself. There is an obvious choice for a discriminator and identifying entity set: the copy number uniquely identifies the BookOnShelf in combination with the corresponding CatalogEntry.

(b)
- Customer(customerID, firstName, lastName, phone)
- Loan(loanID, checkoutDate, scanerID, dueDate, returnDate,
  customerID → Customer, (elimination of *loan customer* table)
  (ISBN, copyNumber) → BookOnShelf) (elimination of *loan copy* table)
- CatalogEntry(ISBN, title, datePublished,
  publisherID → Publisher) (elimination of *publisher of* table)
- BookOnShelf(ISBN → CatalogEntry, (elimination of *copy of* table)
  copyNumber, dateAcquired)
- Author(authorID, firstName, lastName, yearOfBirth)
- Publisher(publisherID, name, phone)
- AuthorOf(ISBN → CatalogEntry, author ID → Author, orderNr)

To enforce the relation cardinalities by design, we have eliminated almost all relationship set tables. The **constraints** are:

- everything **not null**, except for 'returnDate'.

The database schema above enforces all cardinality constraints except for the $1..*$ in the *author of* relationship set. So the design allows a book to have zero authors.